

Sweets Division Problem

Yes. This problem is a **direct application of the Pigeonhole Principle**, and the important idea is to use **prefix sums modulo c**.
 Let's understand it from the beginning.

1. What is the problem asking?

- There are:
- **c** = number of children
 - **n** = number of neighbours
 - **ai** = sweets received from neighbour **i**

The children choose **some subset of neighbours**.
 After collecting sweets, they put everything together and divide equally among **c** children.
 So the total number of sweets must satisfy:

$$\text{total sweets mod } c = 0$$

And every child must get **at least 1 sweet**, so:

$$\text{total sweets} \geq c$$

We need to print **any subset of neighbour indices** whose sweets sum is divisible by **c**.
 If no such subset exists, print:

```
no sweets
```

2. First example

```
4 5
1 2 3 7 5
```

- There are:
- 4 children
 - 5 neighbours

Sweets:

Neighbour	Sweets
1	1
2	2
3	3
4	7
5	5

We need some subset whose sum is divisible by **4**.
 The output is:

```
3 5
```

Neighbour **3** gives **3** sweets.
 Neighbour **5** gives **5** sweets.
 So:

$$3 + 5 = 8$$

Since:

$$8 \text{ mod } 4 = 0$$

The children get:

$$8 / 4 = 2$$

sweets each.

So this is a valid answer.

3. Why does Pigeonhole Principle help?

This is the important part.

We know:

$$n \geq c$$

So there are at least c neighbours.

Let's take the **first c neighbours**.

Calculate their prefix sums.

For the first example:

```
1 2 3 7
```

Prefix sums:

```
1
1 + 2 = 3
1 + 2 + 3 = 6
1 + 2 + 3 + 7 = 13
```

Now take modulo $c = 4$:

```
1 % 4 = 1
3 % 4 = 3
6 % 4 = 2
13 % 4 = 1
```

So:

```
Prefix index: 1 2 3 4
Remainder:   1 3 2 1
```

Notice:

```
remainder 1
```

appears twice.

At prefix 1:

$$S_1 = 1$$

At prefix 4:

$$S_4 = 13$$

Both have the same remainder:

$$S_1 \bmod 4 = S_4 \bmod 4 = 1$$

Therefore:

$$(S_4 - S_1) \bmod 4 = 0$$

And:

$$S_4 - S_1 = 13 - 1 = 12$$

What elements produced 12?

```
2 + 3 + 7 = 12
```

So neighbours:

```
2 3 4
```

are also a valid answer.

Indeed:

$$2 + 3 + 7 = 12$$

and:

$$12/4 = 3$$

So the official output:

3 5

is **not the only answer**.

2 3 4 is also valid.

4. Where exactly is the Pigeonhole Principle?

This is the core.

We have c prefix sums:

$$S_1, S_2, \dots, S_c$$

Each prefix sum has a remainder when divided by c .

Possible remainders are:

$$0, 1, 2, \dots, c - 1$$

There are exactly **c possible remainders**.

So we have:

Pigeons

c prefix sums.

Holes

c possible remainders.

Therefore, one of two things **must happen**.

Case 1: A prefix sum has remainder 0

Suppose:

$$S_i \bmod c = 0$$

Then:

$$S_i$$

itself is divisible by c .

So we simply choose:

1 2 3 ... i

For example:

$c = 4$

$a = 1\ 3\ 2\ 6$

Prefix sums:

1
4
6
12

At prefix 2:

$$4 \bmod 4 = 0$$

Therefore neighbours:

1 2

give:

$$1 + 3 = 4$$

which can be divided equally.

Done.

5. Case 2: No prefix sum has remainder 0

This is where the pigeonhole principle becomes beautiful.

Suppose we have c prefix sums.

And none of them has remainder 0.

Then the only possible remainders are:

1, 2, 3, ..., $c-1$

That's only:

$$c - 1$$

possible remainders.

But we have:

$$c$$

prefix sums.

So:

c prefix sums
↓
 $c - 1$ possible non-zero remainders

By Pigeonhole Principle, **at least two prefix sums must have the same remainder.**

6. Why does same remainder give a valid subset?

Suppose:

$$S_i \bmod c = S_j \bmod c$$

where:

$$i < j$$

For example:

$c = 5$
prefix sum $i = 12$
prefix sum $j = 27$

Their remainders:

$$12 \bmod 5 = 2$$

$$27 \bmod 5 = 2$$

Same remainder.

Now subtract:

$$27 - 12 = 15$$

And:

$$15 \bmod 5 = 0$$

What is $27 - 12$?

It represents exactly the elements **after index i through j .**

So:

$$a_{i+1} + a_{i+2} + \dots + a_j$$

is divisible by c .

Therefore, we found our subset.

7. Visual way to remember it

Think of the remainders as boxes:

POSSIBLE REMAINDERS

0	1	2	3	...	c-1
---	---	---	---	-----	-----

We put every prefix sum into its remainder box.

For c prefix sums:

```

Prefix sums
  ↓
S1 S2 S3 S4 ... Sc
  ↓
remainders
  ↓
 1  3  2  1

```

Here:

```

S1 → remainder 1
S4 → remainder 1

```

Two pigeons entered the same hole.

Therefore:

$$S_4 - S_1$$

is divisible by c .

8. Why do we only need the first c elements?

This is another important observation.

The problem gives:

$$c \leq n$$

Therefore, there are at least c neighbours.

We only need the first c .

Why?

Because among those c elements, the pigeonhole argument **guarantees** that a valid subset exists.

So even if:

```

n = 100000
c = 5

```

we only need to inspect the first:

5 elements

to guarantee an answer.

We don't need to search all possible subsets.

9. Why brute force doesn't work

Suppose $n = 100000$.

Every neighbour can either be:

```

selected
not selected

```

So there are:

$$2^n$$

possible subsets.

That's impossible to check.

The pigeonhole principle gives us a guaranteed answer using only:

$$O(c)$$

work.

10. Complete algorithm

We maintain:

```
C++
rem[r] = prefix index where remainder r was first seen
```

Initially:

```
C++
rem[0] = 0;
```

Then calculate prefix sums for the first c elements.

For every index i :

```
C++
sum += a[i];
r = sum % c;
```

If $r == 0$

Then:

```
1 ... i
```

is our answer.

Otherwise, if remainder r appeared before

Suppose it appeared at index j .

Then:

```
prefix[i] % c == prefix[j] % c
```

Therefore:

```
prefix[i] - prefix[j]
```

is divisible by c .

So choose:

```
j+1 ... i
```

Otherwise

Store:

```
C++
rem[r] = i;
```

11. Applying it to the first example

```
c = 4
a = [1, 2, 3, 7, 5]
```

We examine first 4:

```
i = 1
sum = 1
remainder = 1
rem[1] = 1
```

Next:

```
i = 2
sum = 3
remainder = 3
rem[3] = 2
```

Next:

```
i = 3
sum = 6
remainder = 2
rem[2] = 3
```

Next:

```
i = 4
sum = 13
remainder = 1
```

But:

```
rem[1] = 1
```

already exists.

Therefore:

```
j = 1
i = 4
```

Answer:

```
j+1 ... i
2 3 4
```

Sum:

$$2 + 3 + 7 = 12$$

and:

$$12 \bmod 4 = 0$$

Valid.

12. Second example

```
3 6
7 11 2 5 13 17
```

There are 3 children.

Output:

```
2 3 4
```

Those neighbours give:

```
11 + 2 + 5 = 18
```

Since:

$$18 \bmod 3 = 0$$

each child gets:

$$18/3 = 6$$

So it's valid.

Again, there could be other valid answers.

13. The main pattern you should remember

This problem teaches a **very important competitive-programming pattern**:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

If you need a subset whose sum is divisible by k , look at prefix sums modulo k .

The key theorem is:

```
k prefix sums
  ↓
k possible remainders
  ↓
either one remainder is 0
  OR
two remainders are equal
  ↓
valid subarray/subset
```

And the mathematical reason is:

$$S_j \equiv S_i \pmod{k}$$

therefore:

$$S_j - S_i \equiv 0 \pmod{k}$$

This same **prefix-sum + modulo + pigeonhole** idea appears in many Codeforces/LeetCode problems, especially problems asking for a **contiguous subarray whose sum is divisible by k** .

Sure. This is the clean **$O(c)$** solution using **prefix sum + remainder + Pigeonhole Principle**.

```
C++
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int c, n;

    while (cin >> c >> n) {
        if (c == 0 && n == 0)
            break;

        vector<int> a(n + 1);

        for (int i = 1; i <= n; i++) {
            cin >> a[i];
        }

        // rem[r] = index where remainder r was first seen
        vector<int> rem(c, -1);

        // Prefix sum = 0 before taking any element
        rem[0] = 0;

        long long sum = 0;

        bool found = false;

        // We only need first c elements
        for (int i = 1; i <= c; i++) {
            sum += a[i];

            int r = sum % c;

            // Case 1: prefix sum itself is divisible by c
            if (r == 0) {
                for (int j = 1; j <= i; j++) {
                    cout << j << " ";
                }
                cout << '\n';

                found = true;
                break;
            }

            // Case 2: same remainder appeared before
            if (rem[r] != -1) {
                int start = rem[r] + 1;
                int end = i;

                for (int j = start; j <= end; j++) {
                    cout << j << " ";
                }

                cout << '\n';

                found = true;
                break;
            }

            // First time seeing this remainder
            rem[r] = i;
        }

        if (!found) {
            cout << "no sweets\n";
        }
    }
}
```

```
    return 0;
}
```

The important part

This is the core:

```
C++
sum += a[i];
int r = sum % c;
```

Then:

```
C++
if (r == 0)
```

means the current prefix itself is divisible by c .

Otherwise:

```
C++
if (rem[r] != -1)
```

means we have seen the **same remainder before**.

For example:

```
prefix[i] % c = 2
prefix[j] % c = 2
```

Therefore:

```
(prefix[i] - prefix[j]) % c = 0
```

So the elements:

```
j + 1 ... i
```

form our answer.

Why $\text{rem}[0] = 0$?

This is a very important trick:

```
C++
rem[0] = 0;
```

It means:

"Before taking any element, the prefix sum is 0."

So if the first i elements have sum divisible by c , we can treat it as:

```
prefix[i] % c == prefix[0] % c
```

and output:

```
1 ... i
```

Complexity

We only process the first c elements:

$$O(c)$$

Memory:

$$O(c)$$

The **Pigeonhole Principle** guarantees that an answer exists among the first c neighbours, because $c \leq n$.

Absolutely. Let's dry-run the **exact code** using the first test case:

```
4 5
1 2 3 7 5
```

So:

- $c = 4 \rightarrow 4$ children
- $n = 5 \rightarrow 5$ neighbours
- $a = [1, 2, 3, 7, 5]$

Our code only needs to check the **first $c = 4$ elements**.

1. Initial setup

We read:

```
C++
cin >> c >> n;
```

So:

```
c = 4
n = 5
```

Then:

```
C++
vector<int> a(n + 1);
```

We store:

```
index:  1  2  3  4  5
a[i]:   1  2  3  7  5
```

Then:

```
C++
vector<int> rem(c, -1);
```

Since $c = 4$, we have:

```
rem[0] = -1
rem[1] = -1
rem[2] = -1
rem[3] = -1
```

Then:

```
C++
rem[0] = 0;
```

So:

```
rem[0] = 0
rem[1] = -1
rem[2] = -1
rem[3] = -1
```

What does this mean?

`rem[r]` tells us:

At which prefix index did we first see remainder r ?

And `rem[0] = 0` means:

Before taking any element, prefix sum is 0.

2. Start the loop

Our loop is:

```
C++
for (int i = 1; i <= c; i++)
```

Since:

```
c = 4
```

we will process:

```
i = 1
i = 2
i = 3
i = 4
```

Iteration 1 $\rightarrow i = 1$

Current element:

```
a[1] = 1
```

We execute:

```
C++
sum += a[i];
```

Initially:

```
sum = 0
```

So:

```
sum = 0 + 1
      = 1
```

Now:

```
C++
int r = sum % c;
```

Therefore:

```
r = 1 % 4
   = 1
```

So we have:

```
sum = 1
r = 1
```

Check 1

```
C++
if (r == 0)
```

Is:

```
1 == 0 ?
```

No.

So continue.

Check 2

```
C++
if (rem[r] != -1)
```

That means:

```
C++
if (rem[1] != -1)
```

Currently:

```
rem[1] = -1
```

So:

```
-1 != -1
```

is false.

So this is the **first time** we have seen remainder 1.

We execute:

```
C++
rem[r] = i;
```

Therefore:

```
rem[1] = 1
```

Our array becomes:

```
remainder:  0   1   2   3
            ↓   ↓   ↓   ↓
rem:        0   1  -1  -1
```

Meaning:

```
remainder 0 → seen at prefix 0
remainder 1 → seen at prefix 1
```

Iteration 2 → i = 2

Current element:

```
a[2] = 2
```

Update sum:

```
C++
sum += a[i];
```

Previous:

```
sum = 1
```

Therefore:

```
sum = 1 + 2
     = 3
```

Calculate remainder:

```
C++
r = sum % c;
```

So:

```
r = 3 % 4
  = 3
```

Now:

```
sum = 3
r = 3
```

Check 1

```
C++
if (r == 0)
```

Is:

```
3 == 0?
```

No.

Check 2

```
C++
if (rem[r] != -1)
```

We check:

```
rem[3]
```

Currently:

```
rem[3] = -1
```

So we haven't seen remainder 3.

Store its position:

```
C++
rem[3] = 2;
```

Now:

```
remainder:  0  1  2  3
rem:        0  1 -1  2
```

Iteration 3 $\rightarrow i = 3$

Current element:

```
a[3] = 3
```

Update:

```
sum = 3 + 3
     = 6
```

Calculate remainder:

```
r = 6 % 4
  = 2
```

So:

```
sum = 6
r = 2
```

Check 1

```
C++
if (r == 0)
```

Is:

```
2 == 0?
```

No.

Check 2

```
C++
if (rem[2] != -1)
```

Currently:

```
rem[2] = -1
```

So remainder 2 has never appeared.

Store:

```
C++
rem[2] = 3;
```

Now:

```
remainder: 0 1 2 3
rem:      0 1 3 2
```

Iteration 4 $\rightarrow i = 4$

Current element:

```
a[4] = 7
```

Update sum:

```
sum = 6 + 7
     = 13
```

Calculate remainder:

```
r = 13 % 4
  = 1
```

So:

```
sum = 13
r = 1
```

Now the important moment 🚨

Check:

```
C++
if (r == 0)
```

Is:

```
1 == 0?
```

No.

Now:

```
C++
if (rem[r] != -1)
```

means:

```
C++
if (rem[1] != -1)
```

Look at our `rem` array:

```
remainder: 0 1 2 3
rem:      0 1 3 2
           ↑
```

We already saw remainder 1 at:

```
rem[1] = 1
```

Therefore:

```
rem[1] != -1
```

is **true**.

We found a repeated remainder!

3. Why is this useful?

We have:

```
prefix sum at index 1 = 1
prefix sum at index 4 = 13
```

Their remainders:

```
1 % 4 = 1
13 % 4 = 1
```

Same remainder!

Therefore:

$$13 - 1 = 12$$

And:

$$12\%4 = 0$$

So the elements **between those two prefix sums** must have a sum divisible by 4.

Those elements are:

```
index 2 → 2
index 3 → 3
index 4 → 7
```

Therefore:

```
2 3 4
```

Their sum:

$$2 + 3 + 7 = 12$$

And:

$$12/4 = 3$$

So every child gets:

```
3 sweets
```

Perfect.

4. How does the code find 2 3 4?

We have:

```
C++
int start = rem[r] + 1;
int end = i;
```

Here:

```
rem[r] = rem[1] = 1
i = 4
```

Therefore:

```
start = 1 + 1
      = 2
end = 4
```

So:

```
C++
for (int j = start; j <= end; j++)
```

becomes:

```
j = 2
j = 3
j = 4
```

And it prints:

```
2 3 4
```

5. Complete dry-run table

This table makes the entire algorithm easier to remember:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

i	a[i]	Prefix Sum	sum % 4	rem[r] before	Action
1	1	1	1	-1	Store rem[1] = 1
2	2	3	3	-1	Store rem[3] = 2
3	3	6	2	-1	Store rem[2] = 3
4	7	13	1	1	Same remainder found → answer

At i = 4:

```
prefix[1] = 1
prefix[4] = 13

1 % 4 = 1
13 % 4 = 1
```

Therefore:

```
prefix[4] - prefix[1]
= 13 - 1
= 12
```

and:

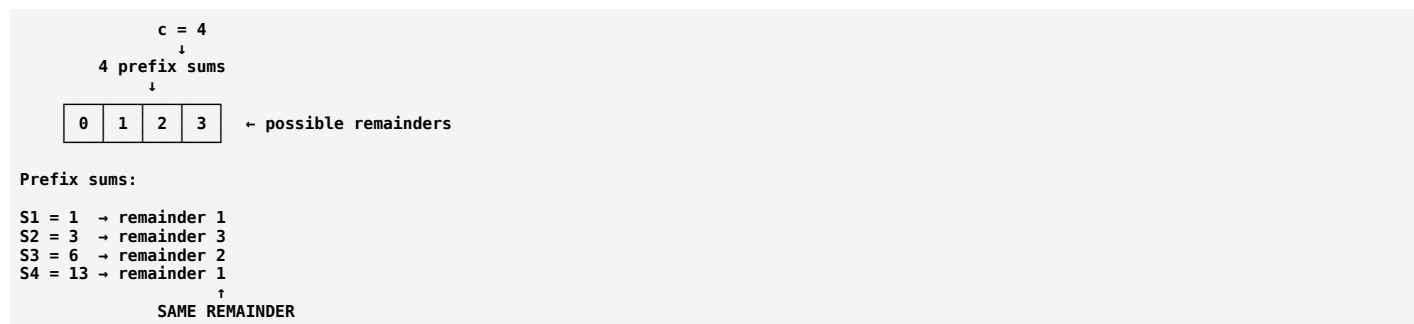
```
12 % 4 = 0
```

Hence:

```
2 3 4
```

6. Visualizing the Pigeonhole Principle here

This is exactly where the **Pigeonhole Principle** appears:



We have:

```
4 prefix sums
```

and:

```
4 possible remainders
```

In this particular case, remainder 1 repeats.

And when two prefix sums have the same remainder:

$$S_j \bmod c = S_i \bmod c$$

then:

$$(S_j - S_i) \bmod c = 0$$

which gives us our required subset.

One very important thing to remember

Don't think:

"Same remainder means the two elements are equal."

✗ That's not what it means.

It means:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

The difference between the two prefix sums is divisible by c .

For this example:

```
13 % 4 = 1
1 % 4 = 1
```

but:

```
13 != 1
```

Their **difference** is:

```
13 - 1 = 12
```

and 12 is divisible by 4.

That's the exact reason the elements **between those two prefix positions** form our answer.

ChatGPT can make mistakes. Check important info.